

Article

YOLO11-FGA: Express Package Quality Detection Based on Improved YOLO11

Peng Zhao ^{1,*}, Guanglei Qiang ¹, Yangrui Fan ¹, Yu Du ¹, Junye Yang ¹ and Zhen Tian ² 

¹ School of Computer Science and Technology, Taiyuan Normal University, Jinzhong 030619, China; 202325502004@stu.tynu.edu.cn (G.Q.); 202325502027@stu.tynu.edu.cn (Y.F.); 202325503001@stu.tynu.edu.cn (Y.D.); yangjunye2025@163.com (J.Y.)

² James Watt School of Engineering, University of Glasgow, Glasgow G12 8QQ, UK; 2620920z@student.gla.ac.uk

* Correspondence: zhaopeng@tynu.edu.cn;

Abstract

In response to the rapidly increasing volume of express parcels, the challenges of insufficient detection accuracy and slow response speed in express parcel quality detection systems have become prominent. To address these issues, we propose the YOLO11-FGA algorithm, an enhanced version of the YOLO11n model designed to improve both detection accuracy and response speed. The key innovation in this model is the introduction of the FasterNet backbone, which enhances the feature extraction capabilities while maintaining a lightweight design, thus improving overall computational efficiency. Additionally, we incorporate the C3k2_GhostDynamicConv module, which combines the GhostBottleneck and DynamicConv structures to significantly enhance the detection and feature extraction capabilities, particularly for subtle and complex packaging defects. To further improve detection performance, an Auxiliary training head (Aux) is added to the YOLO11 detection head, enhancing multi-scale feature fusion and boosting the accuracy of small target detection, which is essential for identifying minor defects. Experimental results demonstrate that YOLO11-FGA outperforms YOLO11n, with improvements in accuracy (1.1%), recall (1.3%), mAP@0.5 (2.3%), and mAP@0.5:0.95 (1.4%). These results highlight the superior performance of the YOLO11-FGA algorithm, which offers enhanced detection accuracy, robustness, and computational efficiency, making it a highly effective solution for real-time express parcel quality detection in logistics applications.

Keywords: YOLO11; logistics package; FasterNet; GhostDynamicConv



Academic Editor: Gholamreza Anbarjafari (Shahab)

Received: 21 October 2025

Revised: 20 November 2025

Accepted: 21 November 2025

Published: 23 November 2025

Citation: Zhao, P.; Qiang, G.; Fan, Y.; Du, Y.; Yang, J.; Tian, Z. YOLO11-FGA: Express Package Quality Detection Based on Improved YOLO11. *Information* **2025**, *16*, 1021. <https://doi.org/10.3390/info16121021>

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

1. Introduction

The logistics industry plays a pivotal role in driving economic development and maintaining the efficiency of social operations. In recent years, with the rapid advancement of e-commerce and the continuous expansion of logistics networks, the handling, transportation, and management of express parcels have become essential components of modern supply chain systems [1–3]. The express delivery sector, in particular, has experienced exponential growth, leading to increased demand for automation, intelligent supervision, and real-time quality monitoring. However, while logistics efficiency continues to improve, the issue of package quality control has emerged as a new challenge within the logistics process.

From the consumer's perspective, parcel packaging integrity, labeling accuracy, content protection, and the absence of external damage have become key indicators for eval-

uating logistics service quality. Customers expect parcels to be delivered quickly while maintaining their original condition, as damage to packaging or contents directly affects the perceived reliability of logistics providers. From the enterprise perspective, logistics companies must ensure high delivery efficiency while maintaining consistent package quality, which is essential for building brand reputation, enhancing customer satisfaction, and improving market competitiveness [4,5]. Consequently, intelligent parcel inspection systems have become increasingly important in ensuring that express deliveries meet both safety and quality standards.

In recent years, deep learning-based visual detection algorithms have made remarkable progress in various fields of industrial quality inspection and defect recognition. Among these, object detection models—particularly the YOLO (You Only Look Once) series—have shown excellent performance in achieving real-time detection and high accuracy, making them suitable for large-scale logistics applications. However, despite their advantages, existing detection models still face several limitations when applied to express package quality inspection. Current deep learning algorithms often exhibit insufficient detection accuracy, struggle to recognize small-scale or partially occluded defects, and perform inconsistently in complex or dynamic environments such as uneven lighting, reflective materials, or irregular surfaces. These shortcomings restrict the deployment of existing models in real-world logistics scenarios that demand both high accuracy and efficiency.

To address the above challenges, this paper proposes an improved model, termed YOLO11-FGA, based on the YOLO11n framework. The YOLO11-FGA model introduces several targeted optimizations to enhance feature extraction efficiency, improve detection precision, and strengthen multi-scale feature perception in express package quality detection tasks. The main contributions of this work are summarized as follows:

1. To address the issue of feature differences in logistics package quality in images, the FasterNet backbone network is used to replace the YOLO11 backbone network to improve feature extraction capabilities and lightweight design.
2. To fully perceive global information and accurately capture the local features of small targets, the C3k2_GhostDynamicConv module is proposed by integrating GhostBottleneck and DynamicConv into C3k2 to improve feature extraction capabilities and package quality detection capabilities.
3. An auxiliary training head Aux is added to the original YOLO11 detection head to enhance the multi-scale feature fusion capability, thereby improving the accuracy of small target detection.

Through the above improvements, the proposed YOLO11-FGA algorithm achieves a balanced integration of speed, accuracy, and robustness, making it highly suitable for real-time express package quality inspection. Experimental results demonstrate that the model significantly improves detection performance, effectively identifying defects such as wrinkles, cracks, tears, and surface deformations under diverse lighting and background conditions. The YOLO11-FGA framework thus provides a technical foundation for intelligent logistics.

2. Related Works

Traditional parcel inspection methods primarily rely on manual visual inspection or classical image-processing-based automated systems, which often suffer from low detection accuracy, limited robustness, and poor adaptability in complex logistics scenarios. With the rise of deep learning and computer vision technologies, object detection models have achieved remarkable breakthroughs, providing new technical pathways for express package quality inspection. Existing object detection methods can be divided into two main categories: two-stage detection frameworks (e.g., Faster R-CNN) and single-stage detection

frameworks (e.g., YOLO series). The two-stage methods achieve high accuracy but require heavy computation due to the region proposal process, while single-stage models like YOLO adopt an end-to-end detection mechanism that simultaneously performs localization and classification within a single forward pass, offering significant computational efficiency and better real-time performance—an essential advantage for logistics quality control systems.

The research on express package quality assessment has attracted increasing attention in recent years, especially with the rapid development of e-commerce logistics and intelligent inspection technologies. Ensuring the integrity, safety, and surface quality of express packages has become a key factor in maintaining logistics efficiency and enhancing customer satisfaction. In this context, researchers have proposed various detection methods and models to improve detection robustness, generalization, and adaptability.

For instance, Lee et al. [6] developed a damage detection technique using ultrasonic guided waves combined with outlier detection to diagnose sealant delamination in integrated circuit (IC) packages. Validated through dye-and-pry tests, their method demonstrated the reliability of multivariate Gaussian models in enhancing defect detection robustness. Extending this direction, Liu et al. [7] introduced a latent diffusion-based data augmentation framework for supervised graph outlier detection, employing a variational encoder to unify heterogeneous graph data into a latent space. This method significantly improved the recognition of structural anomalies and provided theoretical support for model robustness in logistics inspection scenarios. In parallel, Arnaudon et al. [8] proposed PyGenStability, a Python package for multiscale community detection using generalized Markov stability, offering a framework for identifying stable structures in complex networks. Similarly, Almeida-Silva et al. [9] developed the cogeqc R/Bioconductor package, which evaluates the quality of comparative genomics data—such as genome assembly and synteny detection—illustrating the importance of parameter assessment and consistency metrics for ensuring data reliability in automated quality evaluation.

From a manufacturing standpoint, Cao et al. [10] introduced an automatic chip package surface defect detection system that integrates deep learning with attention mechanisms (convolutional block attention module, CBAM, receptive field block, RFB) and clustering algorithms (k-means++). Their introduction of the confidence propagation cluster (CP-Cluster) for non-maximum suppression improved both detection accuracy and confidence levels, providing valuable insights for defect identification in package inspection tasks. Later, in 2024, they further refined the framework in the ACM Transactions on Mathematical Software publication [11], enabling improved graph partition optimization across multiple resolutions.

In recent years, YOLO-based models have gained widespread attention as powerful frameworks for real-time package quality detection. Mao et al. (2024) [12] demonstrated the efficiency of YOLO in detecting defects on dual in-line packages (DIP), combining digital camera optics with deep learning to achieve high accuracy in automated defect recognition. Similarly, Pham and Chang (2023) [13] developed a YOLO-based real-time defect detection system capable of automating packaging inspection, significantly reducing manual intervention and increasing production throughput. Chomklin et al. (2024) [14] performed a comparative analysis of YOLOv8, YOLOv9, and YOLOv10, confirming their superior precision and efficiency in lean manufacturing environments. Dong et al. (2024) [15] proposed AMC-YOLO, an improved YOLOv8 variant that enhances defect detection in cigarette packaging through optimized attention modules and anchor refinement strategies, achieving notable gains in small-object recognition. Moreover, Chen and Shiu (2022) [16] demonstrated YOLO's practical industrial adaptability by applying YOLO-family algorithms to classify product quality in ABS metallization processes, reinforcing YOLO's

scalability across different manufacturing domains. The detailed job description is shown in Table 1.

Table 1. Comparison of Related Research on Package Quality Detection.

Authors	Year	Improvement Method	Core Mechanism	Limitations
Lee H [6]	2021	ultrasonic guided waves and outlier detection	Uses ultrasonic guided waves (UGW) for non-destructive evaluation (NDE) of sealant delamination in integrated circuit (IC) packages, combined with DBSCAN for damage classification.	Limited to delamination detection
Liu K [7]	2023	Data augmentation for supervised graph outlier detection	GODM uses latent diffusion models to generate synthetic outliers, addressing class imbalance in supervised graph outlier detection.	Limited to graph data; does not address other types of data imbalance.
Arnaudon A [8]	2023	PyGenStability for multiscale community detection	Integrates generalized Markov stability to optimize graph partitions at different resolutions.	Requires heavy computational resources for large-scale graphs.
Almeida-Silva F [9]	2023	R/Bioconductor package for assessing comparative genomics data quality	cogeqc package provides tools for evaluating genome assembly and annotation quality, orthogroup inference, and synteny detection, using comparative statistics and graph-based analysis.	Focused on comparative genomics; may not generalize to other biological datasets.
Cao Y [10]	2023	Real-time chip package surface defect detection based on YOLOv7	Proposes a real-time defect detection method using YOLOv7, incorporating k-means++ for anchor frame clustering, CBAM, RFB, and a new confidence propagation cluster (CP-Cluster) to improve accuracy and efficiency.	Limited to chip package defects; not applicable to other domains.
Arnaudon A [11]	2024	PyGenStability for multiscale community detection	Introduces PyGenStability, a Python package for unsupervised multiscale community detection in graphs, using the generalized Markov Stability quality function with the Louvain or Leiden algorithms.	Limited to graph-based applications; may require tuning for specific graphs.
Mao W [12]	2024	Automated defect detection for DIP using YOLO and ConSinGAN	Proposes an automated defect detection system for dual in-line package (DIP) components using YOLO object detection models (v3, v4, v7, v9) combined with ConSinGAN for data augmentation.	Limited to DIP defects; the model's performance may not generalize well to other components.
Pham D L [13]	2023	YOLO-based real-time packaging defect detection system	Proposes a real-time defect detection system based on the YOLO algorithm to automatically classify defective packaged products in industrial quality control.	Limited to packaging defects; may not generalize to all types of defects.
Chomklin A [14]	2024	Comparative study of YOLOv8, YOLOv9, and YOLOv10 for packaging defect detection	Compares various YOLO models for detecting packaging defects in lean manufacturing, utilizing a dataset with seven classes	Limited to packaging defects; dataset may not cover all defect types.
Dong P [15]	2024	AMC-YOLO: Improved YOLOv8-based defect detection for cigarette packs	Proposes AMC-YOLO, a YOLOv8-based model designed for cigarette pack defect detection, incorporating Adaptive Spatial Weight Perception (ASWP), MARFPN, and Cross-Layer Collaborative Detection Head (CLCDH) for improved feature learning and accuracy.	Focused specifically on cigarette packaging defects; may not generalize to other industries.
Chen Y W [16]	2022	YOLO-family algorithms for product quality classification in ABS metallization	Applies YOLO-family algorithms (v2 to v5) to develop an automatic optical inspection (AOI) system for defect detection on reflective surfaces of electroplated Acrylonitrile Butadiene Styrene (ABS) products.	Focused on ABS electroplating defects; may not generalize to other materials or industries.

Collectively, these studies—from traditional signal analysis to deep learning-based object detection—illustrate the evolution of quality inspection techniques. Traditional methods [6] ensure interpretability but lack scalability; graph-based and statistical frameworks [7–9,11] emphasize robustness and data integrity yet are limited in spatial perception; deep learning models [10–16], particularly YOLO-based architectures, achieve end-to-end, high-speed, and high-accuracy defect detection, making them the dominant approach for express package quality assessment. By incorporating lightweight backbones, attention mechanisms, and multi-scale fusion modules, modern YOLO variants offer strong real-time performance and adaptability, paving the way for intelligent, automated, and high-efficiency logistics inspection systems in the era of smart logistics.

3. Materials and Methods

As one of the most recent developments in the YOLO family, YOLOv11n offers significant advancements in real-time object detection, particularly in accurately identifying small-scale defects on complex surfaces. To evaluate its effectiveness, the model was trained and tested on a custom dataset of express package images to detect surface quality issues such as wrinkles, tears, and deformations. YOLOv11n formulates the quality detection task as a regression-based learning problem, employing an enhanced convolutional neural network architecture capable of simultaneously predicting defect categories and bounding box coordinates in a single forward inference pass. This design achieves an effective balance between detection accuracy and computational efficiency, making it highly suitable for industrial quality inspection scenarios.

The choice of YOLOv11n as the foundational model was driven by several key advantages. First, it excels in real-time performance, providing fast and precise inference results that are essential for online inspection of high-throughput package lines. Furthermore, lightweight techniques such as depthwise separable convolution were integrated to reduce computational overhead and improve inference speed, enabling deployment in edge or resource-constrained environments. In addition, the incorporation of the C2PSA attention mechanism strengthens the model's ability to focus on key defect regions, thereby enhancing localization precision and classification accuracy—particularly valuable for detecting subtle or irregular defects on package surfaces. The C3K2 module further refines shallow feature extraction, improving the understanding of low-level texture and edge patterns that are crucial for identifying surface anomalies.

Overall, YOLOv11n demonstrates strong generalization, robustness, and stability across various industrial inspection tasks, confirming its suitability and effectiveness for express package quality detection and providing a solid technical foundation for subsequent optimization and enhancement.

3.1. Model Improvement

YOLO (You Only Look Once) is a single-stage target detection algorithm based on convolutional neural networks. With its excellent real-time detection capabilities, it has long occupied an important position in application scenarios such as industrial inspection, autonomous driving, and intelligent monitoring. Despite multiple iterations, YOLO still maintains its leading edge. In 2024, Ultralytics launched YOLO11 [17]. This version still consists of three core components in its overall architecture: the backbone network (Backbone), the neck network (Neck), and the detection head (Head). Among them, YOLO11n, as a lightweight version of the series, significantly reduces the number of parameters and computational complexity while maintaining strong target detection capabilities. Its backbone network adopts a multi-layer structure including Conv modules, C3K2 modules, SPPF modules, and C2PSA modules. The C2PSA module introduced after the SPPF module

is used to enhance feature extraction capabilities. At the same time, the C3K2 module, as a new structure evolved from the C2f module, introduces multi-sized convolution kernels and a channel splitting strategy to split the input features into two paths: one path is directly transmitted through standard convolution, and the other path is extracted and fused through multiple C3K or Bottleneck structures. This significantly expands the receptive field and improves feature capture capabilities in complex scenarios. In the detection head, some conventional convolutional layers are replaced with depthwise separable convolutions, which significantly reduces the amount of computation and improves efficiency. However, the YOLO11 model still has some limitations in certain scenarios, such as insufficient sensitivity for detecting extremely small objects and need to improve its robustness in complex backgrounds. Therefore, this paper improves the YOLO11n framework in three aspects: First, it introduces a new backbone network, FasterNet, to enhance feature extraction capabilities. Second, by integrating GhostBottleneck and DynamicConv into C3k2, it proposes the C3k2_GhostDynamicConv module, which comprehensively perceives global information and accurately captures the local features of small objects, improving feature extraction and package quality detection. Finally, an auxiliary training head, Aux, is added to the original YOLO11 detection head to enhance multi-scale feature fusion, thereby improving the accuracy of small object detection. The YOLO11-FGA algorithm model is shown in Figure 1.

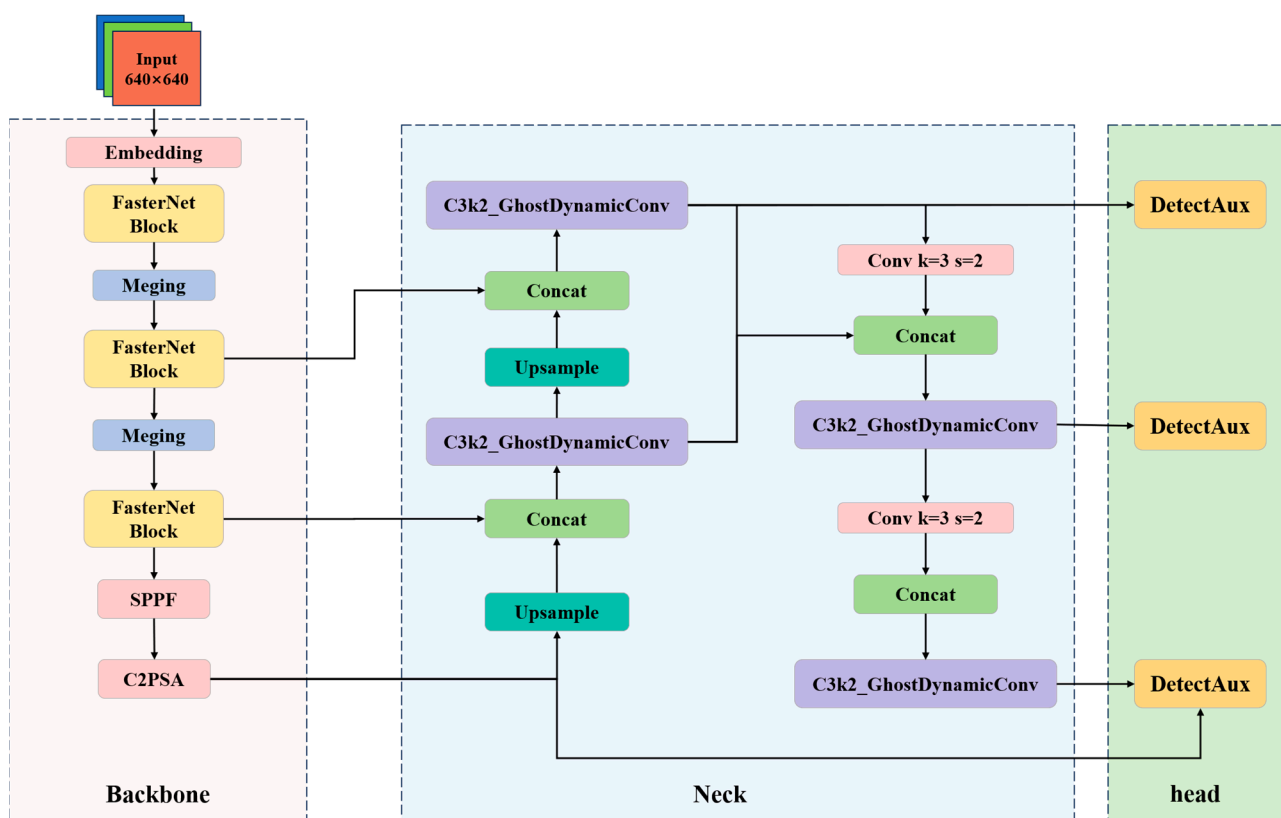


Figure 1. YOLO11-FGA model structure diagram.

3.1.1.1. FasterNet Backbone Network

In the backbone of YOLO11, a series of convolution and deconvolution layers are used, while residual connections and bottleneck structures are used to reduce the size of the network and improve performance. The core is to use C3K2 blocks to handle feature extraction at different stages of the backbone. The smaller 3×3 kernel allows more efficient calculation while retaining the model's ability to capture basic features in the

image. Although the calculation speed has been optimized, its original backbone still contains a lot of computational redundancy, which affects the inference speed. In particular, there may still be latency issues when running on low-computing devices. In order to improve the accuracy of the model, this paper uses the FasterNet network [18] to replace the backbone network of YOLO11n. The FasterNet block mainly consists of a 3×3 PConv and two 1×1 convolutions, which is the core feature extraction module. In the structure of PConv, h , w , and k represent the height, width, and convolution kernel size of the feature map, respectively, and c is the number of channels of the standard convolution. PConv uses standard convolution only on some channels for spatial feature extraction, while the remaining channels remain unchanged, thereby reducing the amount of calculation and memory access overhead while ensuring the consistency of the number of input and output channels. In quality inspection tasks, surface defects (wrinkles, damage, extrusion deformation, scratches, etc.) on express parcels often present as fine-grained, low-contrast, and irregularly shaped local structures. FasterNet's partial convolution strategy concentrates computing power in shallow layers to "scan" key texture channels, while simultaneously preserving global background and appearance consistency through bypassing. 1×1 convolutions then perform cross-channel reconstruction and noise suppression, thus balancing the separability of minor defects with the discriminability of overall appearance. Overall, the FasterNet-based YOLO11n backbone reduces redundant computation, improves feature transfer efficiency and hardware utilization, and significantly improves real-time performance in low-computing scenarios while maintaining or improving detection accuracy. This provides a more stable and efficient basic feature representation for express parcel quality inspection. The FasterNet network architecture is shown in Figure 2.

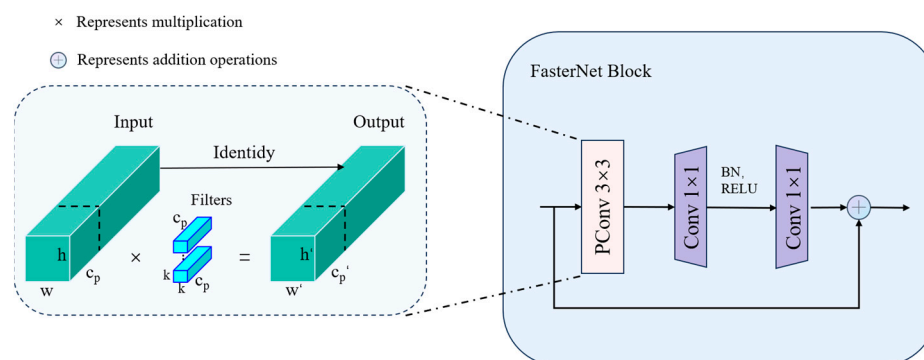


Figure 2. FasterNet Backbone Architecture Diagram.

In this study, FasterNet was chosen as the backbone network due to its excellent balance between accuracy and inference speed. FasterNet achieves high accuracy while exhibiting lower computational complexity and faster inference speed, making it particularly suitable for applications with limited computing resources and high real-time requirements. Through a series of innovative architectural designs, FasterNet optimizes computational efficiency while maintaining high classification performance. Compared to other mainstream backbone networks, FasterNet demonstrates significant advantages in several key evaluation metrics, especially in balancing inference speed and accuracy. Therefore, FasterNet was the preferred backbone network in this study to meet the dual requirements of efficient computation and high accuracy.

In the FasterNet module, PConv 3×3 represents a partial convolution operation, where only a subset of channels of the input feature map are convolved with a 3×3 matrix to reduce computational cost while maintaining efficient feature transfer. Let the size of the input feature map be $h \times w \times c_p$, where h and w represent the height and width, respectively, and c_p is the number of input channels; the kernel size is $k \times k$ ($k = 3$), and the output feature

map is $h' \times \omega' \times c_p'$, where h' and ω' represent the output spatial dimensions, and c_p' is the number of output channels. The “Filters” section in the diagram illustrates this process: PConv performs convolution on a subset of channels, and the channels not involved in the convolution are directly transferred through an identity mapping and added to the convolution result to form the final output.

To verify the advantages of FasterNet over other backbone networks in terms of accuracy and inference speed, comparative experiments were conducted, covering several common and efficient network architectures such as ConvNeXtV2, LskNet, StarNet, and MobileNetV4. The experimental results are shown in Table 2, detailing the performance of each backbone network in terms of accuracy, inference speed, and computational complexity. Experimental results show that FasterNet can significantly improve inference speed while maintaining high accuracy, and its computational resource consumption is low, which fully verifies its superiority in this study.

Table 2. Performance comparison results of different backbones.

Model	P/%	R/%	mAP (0.5)	mAP (0.5:0.95)	FPS
ConvNeXtV2	85.9	81.6	86.7	53.6	60.7
LskNet	85.5	80	87	53.7	57.7
StarNet	85.7	81.1	86.7	54.2	107.8
MobileNetV4	87	80.2	86.8	53.8	99.8
FasterNet	86.4	84	89.1	56.6	125.3

3.1.2. C3k2_GhostDynamicConv

In order to enable the model to better improve the accuracy of express package quality inspection and enhance feature extraction capabilities, this paper integrates GhostBottleneck and DynamicConv into C3k2, thereby proposing the C3k2_GhostDynamicConv model. This model can adaptively integrate different convolution kernels based on input data, dynamically adjust weights, improve the feature extraction capability of express packages, optimize computing resources, and improve the performance of low-precision floating-point operations. Compared with traditional static convolution, DynamicConv [19] has enhanced feature representation capabilities. The principle of DynamicConv is shown in Equations (1) and (2).

$$y = g(\tilde{W}^T(x)x + \tilde{b}(x)) \quad (1)$$

$$\begin{aligned} \tilde{W}(x) &= \sum_{k=1}^k \pi_k(x) \tilde{W}_k, \tilde{b}(x) = \sum_{k=1}^k \pi_k(x) \tilde{b}_k \\ \text{s.t. } 0 &\leq \pi_k(x) \leq 1, \sum_{k=1}^k \pi_k^{k=1}(x) = 1 \end{aligned} \quad (2)$$

where x represents input, y represents output, W , b , and g represent weight, bias, and activation function respectively. $\pi_k(x)$ represents the attention weight of the k -th linear function, and the formula is $\tilde{W}_k^T(x) + \tilde{b}_k$.

DynamicConv consists of k convolutional filters, based on a traditional convolutional neural network (CNN) design. It incorporates batch normalization (BatchNorm) and rectified linear units (ReLU) as activation functions to improve feature expression and computational stability. The specific structure of DynamicConv is shown in Figure 3.

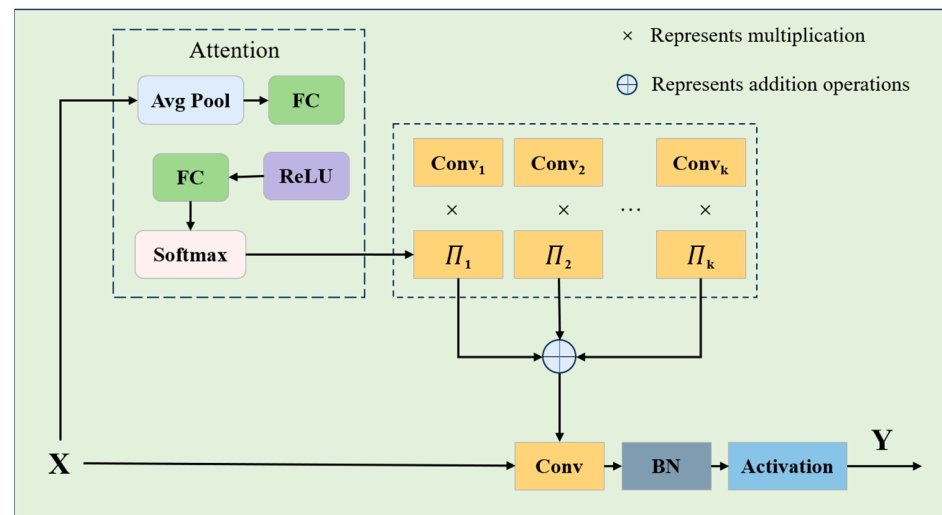


Figure 3. DynamicConv structure.

In DynamicConv, each layer generates k sets of convolutional kernels, whose scale matches the number of channels. These kernels are weighted and combined using corresponding attention weights ($\pi_k(x)$) to form the final convolution parameters for that layer. As shown in the attention module on the left side of Figure 3, $\pi_k(x)$ is calculated as follows. This represents the adaptive weights of the input feature X with respect to the i -th convolutional kernel: First, global average pooling (GAP) is used to extract global spatial features. These features are then mapped to k dimensions through a two-layer fully connected (FC) network and normalized using Softmax. This generates k attention weights, which are used to dynamically adjust the k sets of convolutional kernels in that layer, thereby improving the model's adaptability and feature extraction capabilities. The two modules labeled "FC" in the diagram are actually two cascaded fully connected layers with a sequential connection relationship: the first FC layer is responsible for dimensionality reduction and ReLU activation, while the second FC layer is responsible for restoring the dimensionality and generating attention vectors for Softmax normalization.

In the Ghost module [20], the image first undergoes standard convolution to generate an intrinsic feature map, and then its channels are linearly transformed. This transformation is implemented by deep convolution (DWConv) [21] to generate a Ghost feature map, which is finally concatenated with the intrinsic feature map to form the final output. GhostBottleneck [22] consists of Ghost modules. In the GhostBottleneck model diagram, there are two structures. When the stride is 1, the backbone path consists of two GMs connected in series: the first GM increases the number of channels, and the second GM reduces the number of channels to keep it consistent with the input channel number. The final output result is obtained by adding the intrinsic feature map obtained in the first step and the Ghost feature map obtained in the second step, thereby constructing an efficient deep neural network. The main difference between the two structures is that the depthwise convolution with a stride of 2 is added, which can reduce the spatial size of the feature map to half of the input. The Ghost module and GhostBottleneck structures are shown in Figure 4.

The GhostBottleneck module is composed of Ghost Modules (GM) and has two different structures, as shown in the upper-right corner of Figure 4. From the left diagram in the upper-right corner of Figure 4, it can be seen that when the stride = 1, the main path consists of two consecutive GMs. The first GM increases the number of channels, while the second GM reduces the number of channels to match the input channel count. The intrinsic feature map obtained in the first step is added to the ghost feature map obtained in the second step to

produce the final output, thereby building an efficient deep neural network. The difference between the left and right diagrams in the upper-right corner of Figure 4 is that a depthwise convolution with stride = 2 is inserted between the two GMs in the main path, which reduces the spatial size of the feature map to half of the input. Similarly, the skip connection path also performs the same downsampling to ensure proper alignment for the addition operation. As a result, the final output image resolution is reduced by half.

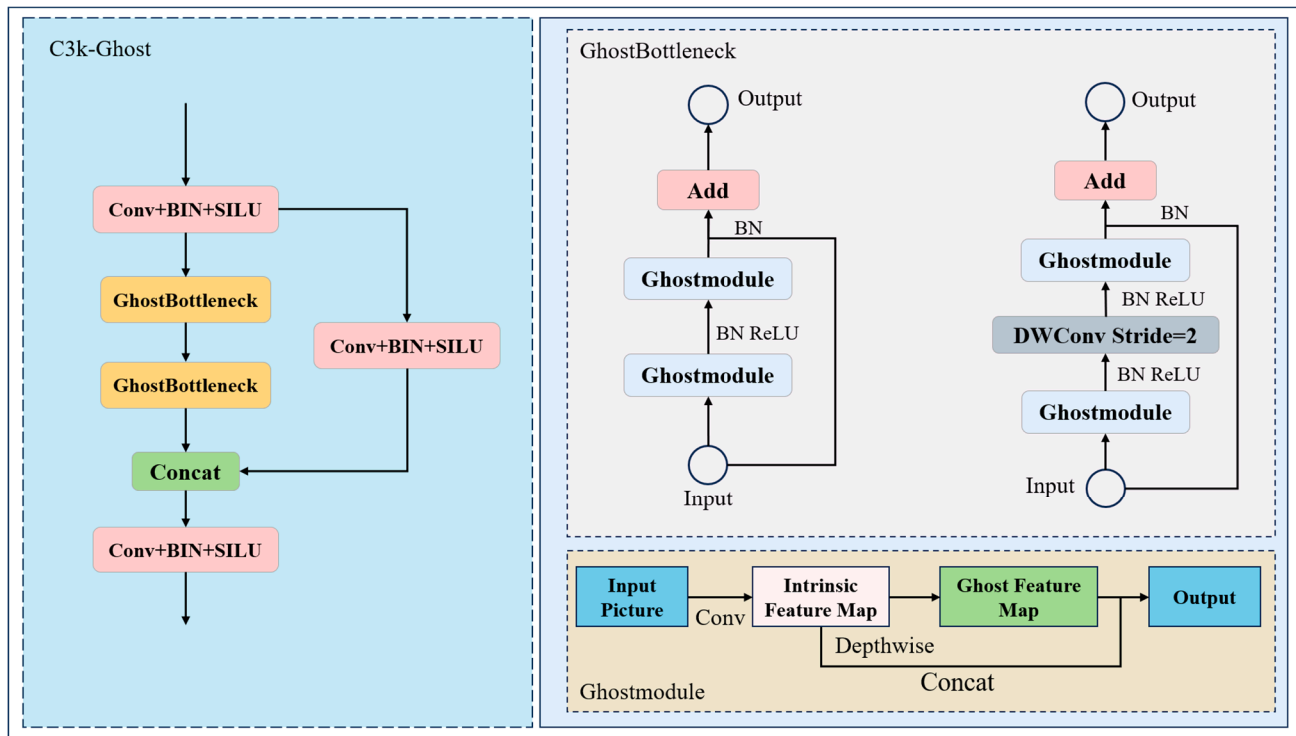


Figure 4. Ghost and GhostBottleneck module structure.

Therefore, the C3k2_GhostDynamicConv model is designed by combining DynamicConv and GhostBottleneck, which is shown in Figure 5. When $c3k = \text{true}$, the residual block is C3k-Ghost, otherwise it is GhostBottleneck.

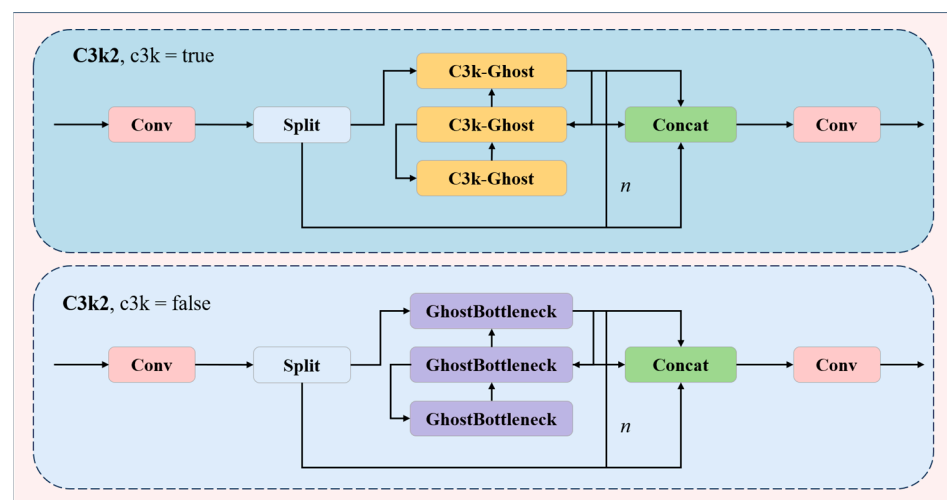


Figure 5. C3k2_GhostDynamicConv structure.

In this model, the value of parameter $c3k$ is pre-set during the model design phase by the network structure configuration process, rather than obtained through training or

hyperparameter search. Its setting follows the trade-off principle between network layer functionality and model size: in shallow or lightweight models, c3k is set to False to reduce computational complexity; in deep or large-scale models, c3k is set to True to enhance feature extraction capabilities.

3.1.3. Auxiliary Detection Head DetectAux

In order to enhance the feature learning and multi-scale feature fusion capabilities of this model during the training phase, an additional auxiliary detection head Aux module is introduced [23]. Due to the diversity of express parcels and the complexity of the scene, such as occlusion, moisture, uneven lighting, and the location of perforations, it is easy to miss image features, thereby affecting the judgment of the quality of the express parcels in the model and thus affecting the success rate of detection. This module participates in training in parallel with the main detection head, improving the convergence and robustness of the feature extraction network by providing additional supervision signals. At the same time, the auxiliary detection head adopts a simplified convolutional structure to focus on capturing fine-grained features. In the inference phase, this auxiliary branch is removed, ensuring that the final model has lower computational overhead and faster real-time response capabilities while maintaining high detection accuracy. The Aux model is shown in Figure 6.

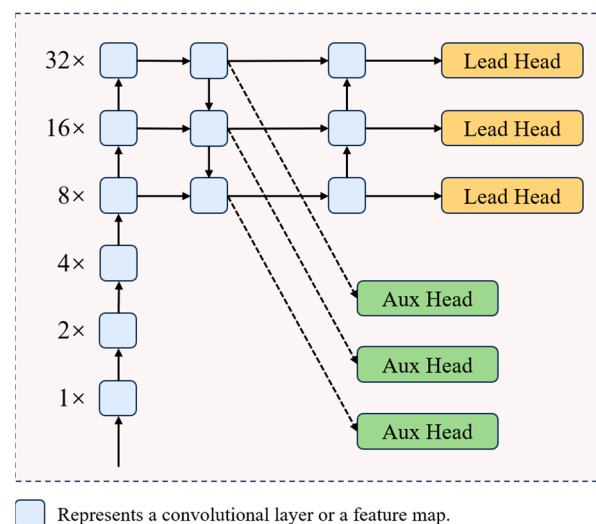


Figure 6. Auxiliary detection head Aux module.

The network incorporates six DetectAux branches to enhance the supervision signal and gradient propagation efficiency of intermediate layer features during training. Specifically, the network simultaneously sets up three lead heads and three aux heads at different scales of the feature pyramid, corresponding to feature layers with different downsampling factors. The lead heads generate the final object detection results, while the aux heads are only activated during training to provide additional supervision for shallow features, enhancing feature representation and accelerating model convergence. During training, all six branches output detection results simultaneously to calculate the joint loss; during inference, only the lead head branch is retained to generate the final detection output.

To enhance the overall expressive capability of the proposed detection network while ensuring deployment efficiency, the structural design follows four key principles: multi-scale feature completion, robustness enhancement, training efficiency optimization, and computational resource control. These principles jointly guide the construction of the DetectAux-enhanced YOLO11 architecture for express package quality inspection.

1. Multi-scale feature enhancement for fine-grained defect detection: Express packages captured in real-world logistics environments exhibit substantial variations in size, surface texture, and viewing angle due to packaging diversity and illumination changes. Traditional detection networks with a limited number of primary detection heads tend to overlook small or subtle defects such as fine wrinkles, scratches, or partial tears. To address this issue, six DetectAux modules are introduced and connected to feature maps at different scales of the backbone network. These additional detection branches enable the model to aggregate multi-scale spatial information, thereby improving the perception of small and irregular defects. By complementing the primary detection head, DetectAux strengthens fine-grained feature representation and ensures more complete multi-scale defect detection.
2. Robustness improvement under complex inspection environments: In practical logistics scenarios, external factors such as overlapping packages, reflective materials, or uneven lighting conditions often interfere with visual perception. To enhance detection stability, the DetectAux module establishes multiple parallel feature pathways that extract discriminative cues from different receptive field perspectives. This redundant supervision mechanism allows the model to maintain high accuracy even under partial occlusion or background clutter, significantly improving the overall robustness and adaptability of the system.
3. Convergence acceleration and supervised reinforcement during training: During the training stage, DetectAux acts as an auxiliary supervision mechanism, providing additional gradient propagation paths that alleviate vanishing gradient issues and improve parameter update efficiency. Each auxiliary head independently computes a local detection loss, which guides the backbone to learn discriminative features more rapidly in the early stages of training. This multi-path supervision not only accelerates convergence but also enhances the overall feature learning quality, allowing the model to achieve optimal detection performance with fewer training epochs.
4. Resource control and computational efficiency optimization: Although six DetectAux branches are introduced, the design remains lightweight and deployment-friendly. Each auxiliary detection head employs small convolutional kernels and reduced channel dimensions, ensuring that the additional parameters contribute minimal computational overhead. The auxiliary branches are active only during training to enrich feature learning, while in the inference phase, all DetectAux modules are removed, retaining only the main detection head. This design ensures real-time inference and efficient deployment on resource-limited platforms such as industrial embedded GPUs. Furthermore, the modular nature of DetectAux supports flexible activation and deactivation, allowing easy adaptation across different hardware environments. Techniques such as mixed-precision training and gradient checkpointing can further reduce memory consumption and improve computational efficiency in large-scale industrial inspection tasks.

In summary, the integration of DetectAux modules provides a balance between model accuracy and inference efficiency. It enhances the YOLO11-based express package quality detection network's ability to capture fine-grained defects, improves robustness in complex industrial conditions, and ensures practical deployability in real-time logistics inspection systems.

4. Data Source and Processing

The dataset used in this article is derived from a self-built express parcel quality inspection sample library, designed to fully reflect the types of common package surface defects encountered during express transportation and warehousing. To ensure data

authenticity and diversity, data collection was conducted under a variety of environmental conditions, including natural light, artificial light, and complex background scenes. The capture equipment included high-definition digital cameras, industrial-grade cameras, and various smartphone models to ensure diverse sample sources and consistent resolution. All images were captured under unified acquisition specifications, maintaining relative stability in parameters such as exposure, focal length, and distance to avoid feature deviations caused by differences in lighting or angle.

The collected data primarily covers three typical types of package surface defects: scratches, wetness, and holes. These three types of defects are common in logistics and comprehensively represent quality issues that may arise during package handling, stacking, and transportation. Tear defects manifest as fiber breakage or interlayer delamination on the carton surface; wetness defects are often caused by rainwater or liquid leakage, resulting in discoloration or wrinkling of the paper material; and holes reflect penetrating damage caused by external impact. By capturing images of various package types (such as corrugated cartons, plastic packaging bags, and composite mailing bags), the dataset fully captures the diversity of materials, textures, and colors.

After data collection was completed, the raw images underwent systematic data cleaning and preprocessing. First, blurred, improperly exposed, or duplicated images were removed to ensure data quality. Second, samples with inconsistent resolutions were resized and converted to the same format. For images with significant background interference, appropriate cropping was performed to highlight the subject area. The cleaned dataset contained 3824 valid samples, ensuring a roughly balanced number across defect categories, thus avoiding bias caused by uneven sample distribution during model training.

To ensure scientific model training and performance evaluation, this paper randomly partitioned the dataset into three subsets in an 8:1:1 ratio: a training set of 3059 images, a validation set of 382 images, and a test set of 383 images. The training set is used to learn model parameters, the validation set is used to adjust hyperparameters and prevent overfitting, and the test set is used for independent evaluation of final performance, ensuring the objectivity and reliability of the model's generalization capabilities. A fixed random seed strategy was used during the partitioning process to ensure reproducibility of experimental results.

Figure 7 shows a sample of the dataset, which allows for a visual analysis of the differences and complexity of different defect types. Overall, this self-built dataset not only covers a wide range of real-world defect forms and shooting scenarios, but also boasts high resolution, clear annotations, and excellent scalability, providing solid data support for subsequent research on intelligent surface quality inspection algorithms for express parcels.

The total dataset contains 3828 images and 6940 annotations, covering three types of defects. To ensure the accuracy and consistency of the annotations, we followed strict annotation protocols and performed multiple rounds of verification. First, all annotations were done by the authors, followed by cross-checking and verification processes to ensure that the boundaries and features of each defect were accurately labeled. Detailed data information is shown in Table 3.

Table 3. Dataset category annotation information.

Category	Number of Pictures	Number of Annotations
scratch	1053	2858
hole	1060	1727
wet	1715	2355
all	3828	6940



Figure 7. Example of express package.

Scratch refers to surface abrasion or linear marks caused by friction or other factors during transportation. This type of defect typically manifests as visible linear or irregular patterns on the packaging surface; the damage is generally shallow. The color of the scratched area differs from the surrounding packaging material, and the surface may change due to material exposure. A hole refers to penetrating damage, usually caused by impact or sharp objects. This type of defect indicates a breach in the integrity of the packaging material, resulting in punctures or tears that may expose the internal contents. The shape of the damage is usually irregular, and its size varies considerably. A wet defect refers to damage to the packaging surface caused by moisture or liquid contamination. Wet damage often manifests as visible water stains or color changes; darker areas indicate that the packaging has absorbed liquid.

When a defect is incorrectly labeled, we still consider it a “defect detected”. Specifically, if the true label is “hole” but it is detected as a “wet” defect, the model successfully identifies the defect area despite the incorrect label, and therefore it is considered a “defect detected.” This is a misclassification, not a missed detection. In our definition, a missed detection means the model completely fails to identify the defect area; that is, no defect is detected. Therefore, a labeling error is not the same as a missed detection; misclassification simply indicates an incorrect category identification of the defect, but the defect itself has been detected.

5. Results and Analysis

In this section, we present a comprehensive evaluation of the proposed YOLO11-FGA model using real-world industrial datasets collected in operational environments. The experiments are structured to rigorously assess the model's performance through a series of controlled tests. Initially, we conduct an ablation study (see Section 5.3) to isolate the contribution of each core component—namely the FasterNet backbone, the C3k2_GhostDynamicConv module, and the auxiliary head (Aux)—by sequentially removing them and measuring the impact on detection performance. This allows us to identify the individual effects of each module on the overall model efficacy. Following this, we compare YOLO11-FGA against several widely adopted YOLO-based models under the same testing conditions (see Section 5.4), with a focus on highlighting its superior performance in terms of detection accuracy, robustness, and adaptability. Finally, we provide an in-depth analysis of the detection results, demonstrating the model's high efficiency and reliability in express package quality detection. To further validate the practical applicability of our approach, we apply both the baseline model and the enhanced EIM-YOLO on real images from industrial production lines, thereby confirming the feasibility and scalability of the model in real-world logistics settings.

5.1. Experimental Environment and Parameter Settings

All experiments in this study were conducted in a Linux-based environment running Ubuntu, utilizing PyTorch 2.2.2 as the deep learning framework with CUDA 12.1 for GPU acceleration to enhance computational performance. To maintain input consistency, all input images were resized to a uniform resolution of 640×640 pixels prior to model entry. This resizing strategy was adopted to achieve an effective trade-off between computational efficiency and feature representation. Each model was trained for 300 epochs with an early stopping policy triggered after 30 epochs of no improvement, ensuring full convergence and stable optimization. The training process was performed on a high-performance system equipped with an NVIDIA RTX 4090 24 GB GPU and an Intel Xeon E5-2680 v4 @ 2.40 GHz CPU, which provided the necessary computational resources for both training and inference. The detailed hardware configuration is presented in Table 4. All operations and tensor computations were executed on CUDA-enabled devices to leverage hardware acceleration. The training hyperparameters included: a dynamically adjusted batch size of 64, an initial learning rate of 0.01, Stochastic Gradient Descent (SGD) optimizer with adaptive learning rate scheduling, and a bounding box scale factor set to 0.7. The random seed was set to 3407 for all experiments to ensure reproducibility. Each experiment was repeated 5 times to account for any variability and ensure the robustness of the results.

Table 4. Hardware configuration of the experimental platform.

Name	Environment Configuration
Operating system	Ubuntu 22.04
CPU	Intel(R) Xeon(R) E5-2680 v4 @ 2.40 GHz
GPU	RTX 4090
GPU graphics memory	24 GB
programming language	Python3.10
Deep Learning Framework	PyTorch2.2.2 and Cuda12.1

To improve training efficiency and reduce the risk of overfitting, we initialized the model with pre-trained weights. This approach enabled the model to leverage generalized feature representations, which helped prevent overfitting, particularly given the small size of our dataset. The performance of our method was compared to existing approaches in the

literature based on standard evaluation metrics such as precision, recall, and mean Average Precision (mAP). The results provide a comprehensive and quantitative assessment of the model's performance, demonstrating notable improvements, especially in small object detection, as reflected in the evaluation metrics mentioned above.

5.2. Evaluation Indicators

In this study, we assess the performance of the proposed YOLO11-FGA model using several standard evaluation metrics that are commonly applied in object detection tasks. These metrics include Precision, Recall, Average Precision (AP), and mean Average Precision (mAP) at different Intersection over Union (IoU) thresholds. These measures provide a comprehensive understanding of the model's ability to detect defects in express package quality detection, balancing the trade-off between detection accuracy and recall. The following formulas describe the evaluation metrics used in this study [24,25].

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

$$R = \frac{TP}{TP + FN} \quad (4)$$

$$AP = \int_0^1 P(R) dR \quad (5)$$

$$mAP_{50} = \frac{1}{N} \sum_{i=1}^N AP_i(IoU \geq 0.5) \quad (6)$$

$$mAP_{50-95} = \frac{1}{N} \sum_{i=1}^N AP_i(IoU \in [0.5, 0.95]) \quad (7)$$

Precision measures the proportion of True Positive detections (TP) out of all detections classified as positive (i.e., both true positives and false positives). In the context of express package quality detection, precision indicates how accurately the model identifies defects in packages, ensuring that the detected defects are truly present without false alarms. A high precision value signifies fewer false positives and a more reliable detection system.

Recall, or sensitivity, is the proportion of True Positive detections (TP) out of all actual positive instances, including both true positives and False Negatives (FN). In express package quality inspection, recall measures the model's ability to detect all relevant defects, including smaller or partially occluded ones. A higher recall value means the model is more sensitive to detecting defects, though it may also result in a higher number of false positives.

Average Precision is a comprehensive measure that integrates Precision and Recall over a range of recall values. It is computed as the area under the precision-recall curve, which illustrates the trade-off between precision and recall at different thresholds. AP gives a single score that reflects the model's performance across all detection thresholds, making it a useful indicator of overall detection accuracy. In the case of package quality detection, it helps evaluate the model's overall effectiveness in identifying defects across a variety of conditions.

mAP50 calculates the mean of Average Precision values when the Intersection over Union (IoU) between predicted and ground truth bounding boxes is greater than or equal to 0.5. This threshold is commonly used to define a successful detection. mAP50 evaluates the model's precision at a moderate overlap requirement and is typically used to assess basic detection performance, focusing on how well the model performs when a sufficient overlap between predicted and true object boundaries exists.

mAP50-95 extends the mAP calculation by averaging the Average Precision across multiple IoU thresholds, from 0.5 to 0.95, in increments of 0.05. This metric provides a more comprehensive evaluation of the model’s ability to detect objects at varying levels of overlap between predicted and ground truth bounding boxes. It is a more stringent measure of detection performance, as it accounts for both high and low overlap situations, making it particularly useful for assessing the model’s robustness in complex real-world environments, where smaller or partially occluded defects may be harder to detect.

These evaluation metrics, particularly mAP50 and mAP50-95, are critical for understanding the accuracy, robustness, and scalability of the YOLO11-FGA model in the context of express package quality detection, ensuring that the model can handle various defect sizes, types, and detection challenges in real-world logistics scenarios.

5.3. Optimizing the Module Ablation Experiment

To better demonstrate the accuracy of express package quality inspection, this model conducted eight ablation experiments. The experimental results are shown in Table 5, using P, R, mAP0.5, mAP0.5:0.95, and GFLOPs as evaluation metrics.

Table 5. Performance comparison of the ablation experiment, “√” indicates that the module is included; “×” indicates that the module is not used.

FasterNet	C3k2_GhostDynamicConv	Aux	P/%	R/%	mAP (0.5)	mAP (0.5:0.95)	GFLOPs (G)	FPS	Params (M)
×	×	×	85.8	81.4	87.7	55.4	6.3	142.7	2.58
√	×	×	86.4	84	89.1	56.6	9.2	125.3	3.90
×	√	×	83.6	83.2	88.3	55.2	5.4	61.7	2.22
×	×	√	86.8	80.6	88.6	56.4	6.3	141	2.58
√	√	×	87.3	81.5	89.1	57.3	8.7	80.7	3.71
√	×	√	86.4	82.9	89.4	58.1	9.2	122.5	3.90
×	√	√	84.4	84.8	89.4	56.5	5.8	89.1	2.39
√	√	√	86.9	82.7	90.0	56.8	8.7	82.7	3.71

GFLOPS (Giga Floating Point Operations Per Second) is a commonly used metric for measuring computer processor performance, representing the number of floating-point operations that can be performed per second. 1 GFLOPS equals 1 billion floating-point operations per second. Floating-point operations handle numerical calculations with decimal parts and are widely used in scientific computing, image processing, machine learning, and other fields. In high-performance computing, GFLOPS is used to evaluate a system’s computational power when performing complex mathematical operations; therefore, this metric reflects the system’s efficiency when performing large-scale data computations. GFLOPS is typically used to evaluate the efficiency and performance of processors in computationally intensive tasks, such as deep learning model training and image rendering. A higher GFLOPS value indicates that the processor can perform more floating-point operations per unit of time, thereby improving the processing speed and efficiency of the task.

Experiments show that after introducing the FasterNet backbone, mAP0.5 increased by 1.4 points, and mAP0.5:0.95 increased by 1.2 points; after adding C3k2_GhostDynamicConv, mAP0.5 increased by 0.6 points; after adding the additional auxiliary detection head Aux, mAP0.5 increased by 0.9 points; after fusing the FasterNet backbone and C3k2_GhostDynamicConv modules, P increased by 1.5 points, mAP0.5 increased by 1.4 points, and mAP0.5:0.95 increased. After fusing the FasterNet backbone and the additional auxiliary head Aux, mAP0.5 increased by 1.7 points; after fusing the C3k2_GhostDynamicConv module and the additional auxiliary head Aux, mAP0.5 increased by 1.7 points; finally, after fusing the FasterNet backbone, the C3k2_GhostDynamicConv module and the additional auxil-

iary head Aux, P increased by 1.1 points, R increased by 1.3 points, mAP0.5 increased by 2.3 points, and mAP0.5:0.95 increased by 1.4 points.

5.4. Comparative Experiment

To verify that YOLO11-FGA can better detect the quality of express packages, a comparative experiment was conducted on different versions of YOLO. P, R, mAP0.5, mAP0.5:0.95, and GFLOPs were used as evaluation indicators. The experimental comparison results are shown in Table 6.

Table 6. Performance comparative results of different models.

Model	P/%	R/%	mAP (0.5)	mAP (0.5:0.95)	GFLOPs (G)
YOLOv5n	85.7	82.1	87.9	52.5	4.1
YOLOv8n	85.7	81.5	88.6	56.4	8.1
YOLOv10n	84.5	76.9	85.9	54.7	8.2
YOLOv11n	85.8	81.4	87.7	55.4	6.3
YOLOv11s	84.8	81.7	87.3	57.2	21.3
RT-DETRr18 [26]	83.4	77.7	83.8	52.0	56.95
PP-YOLOEs [27]	85.4	83.2	87.6	55.9	18.96
Hyper-YOLOt [28]	86.4	82.1	88.2	56.7	8.91
YOLO11-FGA	86.9	82.7	90.0	56.8	8.7

Experimental results show that YOLO11-FGA outperforms other models in all relevant metrics, with improvements of 2.1%, 1.4%, 4.1%, 2.3%, and 2.7% over YOLO5n, YOLO10n, YOLO11n, and YOLO11s, respectively. Therefore, the YOLO11-FGA proposed in this paper can address issues such as insufficient detection accuracy and slow response speed, providing a solution for express package quality inspection.

5.5. Comparison of Model Performance Before and After Improvement

The original YOLOv11 model and the improved YOLO11-FGA model were evaluated using precision and recall, and the results are shown in Figure 8.

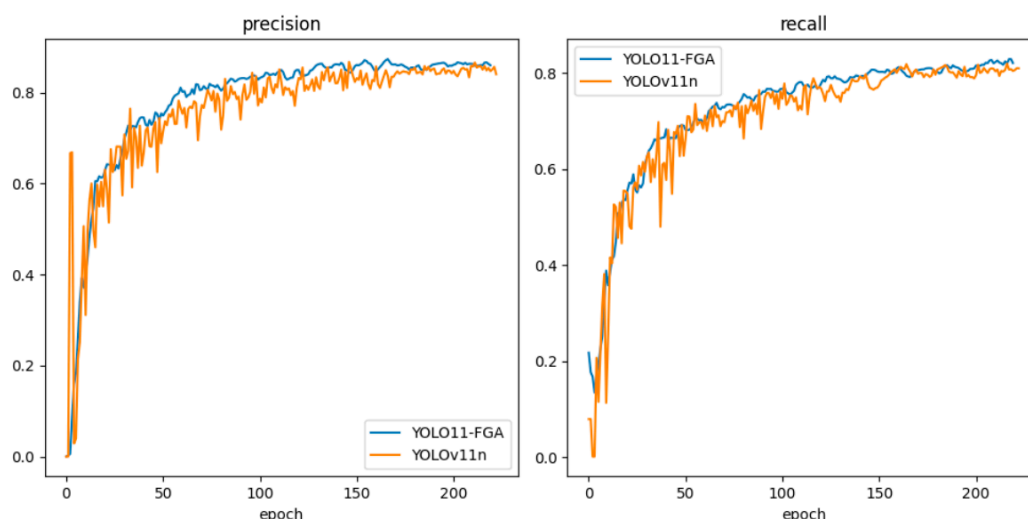


Figure 8. Comparison of Model Performance Before and After Improvement.

The results show that the improved YOLO11-FGA model significantly outperforms the original model in logistics package detection tests. Training curves demonstrate that compared to the baseline model YOLOv11, YOLO11-FGA exhibits a steady improvement in both precision and recall, with the performance advantage becoming more pro-

nounced in the later stages of training. This performance improvement is primarily attributed to the introduction of the FasterNet backbone module and the optimization of the C3k2_GhostDynamicConv module. Although there is a brief performance adaptation period in the early stages of training, thanks to the improvements in these innovative modules, the model converges quickly and remains stable after parameter optimization. This verifies that the improved scheme significantly enhances detection performance while maintaining its lightweight nature, ensuring efficient system response and accuracy.

5.6. Analysis of Test Results

To better demonstrate the performance of our model, we tested the YOLO-FGA and YOLO11n models on a self-built dataset and analyzed the test results visually. Comparing the visualizations reveals that our model outperforms the original model in identifying the quality of express packages. The visualization is shown in Figure 9.



Figure 9. Visual comparison results.

Comparing the visual results shows that our model achieves significantly better accuracy in express package inspection, significantly outperforming the original model. In Figure 9, the original model misses detections, while our model can more comprehensively identify targets. In the express package quality inspection task, our model demonstrates superior detection results.

6. Conclusions

The YOLO11-FGA model proposed in this paper systematically optimizes the traditional object detection framework. By introducing the FasterNet backbone network, the C3k2_GhostDynamicConv module, and the auxiliary detection head (Aux), it significantly improves feature extraction efficiency and detection accuracy while maintaining model lightweight. The FasterNet backbone effectively reduces computational redundancy in the feature extraction stage, enabling high-speed inference even on low-power devices. The C3k2_GhostDynamicConv module combines a dynamic convolution mechanism with

a feature recombination strategy to enhance the network's adaptive perception of subtle surface defects (such as wrinkles, scratches, indentations, and holes) on express packages. The auxiliary detection head optimizes multi-scale feature learning through parallel supervision, accelerating model convergence and improving the stability of small object detection. Experimental results demonstrate that the YOLO11-FGA model demonstrates excellent detection performance on a self-built express package quality inspection dataset. Not only does it achieve significant improvements in precision and recall, but it also maintains a good balance between inference speed and model complexity. It exhibits strong real-time and robustness, and can accurately identify complex defects under varying sizes, lighting, and background conditions. Furthermore, this paper integrates the YOLO11-FGA model with a blockchain-based cold chain logistics traceability system, enabling real-time on-chain uploading and trusted evidence storage of detection results. When an abnormal package is detected, the system automatically generates a corresponding traceability record and writes the detection data, timestamp, and node information to the blockchain ledger. This enables real-time monitoring of logistics nodes and data tamper-proof management, providing strong support for cold chain supply chain security and quality traceability.

In the future, the YOLO11-FGA model will continue to expand its application in smart logistics and intelligent warehousing, and will be deeply integrated with privacy-preserving computing technologies (such as federated learning and homomorphic encryption) to achieve secure cross-enterprise data sharing and collaborative optimization, further improving the overall security and intelligence of the system.

Author Contributions: Conceptualization, P.Z. and G.Q.; methodology, P.Z. and G.Q.; validation, P.Z., Y.F., G.Q. and Y.D.; formal analysis, P.Z.; investigation, Y.F. and J.Y.; resources, Z.T.; data curation, Y.D.; writing—original draft preparation, G.Q.; writing—review and editing, G.Q.; visualization, J.Y.; supervision, P.Z.; project administration, P.Z. and Z.T. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The data presented in this study are available on request from the corresponding author. The data are not publicly available due to privacy and data protection restrictions.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Jazairy, A.; Persson, E.; Brho, M.; von Haartman, R.; Hilletoft, P. Drones in last-mile delivery: A systematic literature review from a logistics management perspective. *Int. J. Logist. Manag.* **2025**, *36*, 1–62. [\[CrossRef\]](#)
2. Chen, L.; Dong, T.; Li, X.; Xu, X. Logistics engineering management in the platform supply chain: An overview from logistics service strategy selection perspective. *Engineering* **2025**, *47*, 236–249. [\[CrossRef\]](#)
3. Jazairy, A.; Pohjosenperä, T.; Prativiera, L.B.; Juntunen, J. Innovators and transformers: Revisiting the gap between academia and practice: Insights from the green logistics phenomenon. *Int. J. Phys. Distrib. Logist. Manag.* **2025**, *55*, 341–360. [\[CrossRef\]](#)
4. Malhotra, G.; Kharub, M. Elevating logistics performance: Harnessing the power of artificial intelligence in e-commerce. *Int. J. Logist. Manag.* **2025**, *36*, 290–321. [\[CrossRef\]](#)
5. Li, C.; Wang, Y.; Li, Z.; Wang, L. Study on calculation and optimization path of energy utilization efficiency of provincial logistics industry in China. *Renew. Energy* **2025**, *243*, 122594. [\[CrossRef\]](#)
6. Lee, H.; Koo, B.; Chattopadhyay, A.; Neerukatti, R.K.; Liu, K.C. Damage detection technique using ultrasonic guided waves and outlier detection: Application to interface delamination diagnosis of integrated circuit package. *Mech. Syst. Signal Process.* **2021**, *160*, 107884. [\[CrossRef\]](#)
7. Liu, K.; Zhang, H.; Hu, Z.; Wang, F.; Yu, P.S. Data augmentation for supervised graph outlier detection with latent diffusion models. *arXiv* **2023**, arXiv:2312.17679. [\[CrossRef\]](#)
8. Arnaudon, A.; Schindler, D.J.; Peach, R.L.; Gosztolai, A.; Hodges, M.; Schaub, M.T.; Barahona, M. PyGenStability: Multiscale community detection with generalized Markov Stability. *arXiv* **2023**, arXiv:2303.05385.

9. Almeida-Silva, F.; Van de Peer, Y. Assessing the quality of comparative genomics data and results with the cogeqc R/Bioconductor package. *Methods Ecol. Evol.* **2023**, *14*, 2942–2952. [[CrossRef](#)]
10. Cao, Y.; Ni, Y.; Zhou, Y.; Li, H.; Huang, Z.; Yao, E. An auto chip package surface defect detection based on deep learning. *IEEE Trans. Instrum. Meas.* **2023**, *73*, 3507115. [[CrossRef](#)]
11. Arnaudon, A.; Schindler, D.J.; Peach, R.L.; Gosztolai, A.; Hodges, M.; Schaub, M.T.; Barahona, M. Algorithm 1044: PyGenStability, a multiscale community detection framework with generalized markov stability. *ACM Trans. Math. Softw.* **2024**, *50*, 1–8. [[CrossRef](#)]
12. Mao, W.-L.; Wang, C.-C.; Chou, P.-H.; Liu, Y.-T. Automated defect detection for mass-produced electronic components based on YOLO object detection models. *IEEE Sens. J.* **2024**, *24*, 26877–26888. [[CrossRef](#)]
13. Pham, D.L.; Chang, T.W. A YOLO-based real-time packaging defect detection system. *Procedia Comput. Sci.* **2023**, *217*, 886–894.
14. Chomklin, A.; Jaiyen, S.; Wattanakitrungron, N.; Mongkolnam, P.; Chaikhan, S. Packaging defect detection in lean manufacturing: A comparative study of yolov8, yolov9, and yolov10. In Proceedings of the 2024 28th International Computer Science and Engineering Conference (ICSEC), Khon Kaen, Thailand, 6–8 November 2024; pp. 1–6.
15. Dong, P.; Wang, Y.; Yu, Q.; Feng, W.; Zong, G. AMC-YOLO: Improved YOLOv8-based defect detection for cigarette packs. *IET Image Process.* **2024**, *18*, 4873–4886. [[CrossRef](#)]
16. Chen, Y.W.; Shiu, J.M. An implementation of YOLO-family algorithms in classifying the product quality for the acrylonitrile butadiene styrene metallization. *Int. J. Adv. Manuf. Technol.* **2022**, *119*, 8257–8269. [[CrossRef](#)]
17. Khanam, R.; Hussain, M. Yolov11: An overview of the key architectural enhancements. *arXiv* **2024**, arXiv:2410.17725. [[CrossRef](#)]
18. Chen, J.; Kao, S.H.; He, H.; Zhuo, W.; Wen, S.; Lee, C.H.; Chan, S.H.G. Run, don't walk: Chasing higher FLOPS for faster neural networks. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Vancouver, BC, Canada, 17–24 June 2023; pp. 12021–12031.
19. Chen, Y.; Dai, X.; Liu, M.; Chen, D.; Yuan, L.; Liu, Z. Dynamic convolution: Attention over convolution kernels. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Seattle, WA, USA, 13–19 June 2020; pp. 11030–11039.
20. Razmand, M. *Development of a Tile Module for Ghost and Its Application*; The University of Iowa: Iowa City, IA, USA, 2020.
21. Gao, H.; Yang, Y.; Li, C.; Gao, L.; Zhang, B. Multiscale residual network with mixed depthwise convolution for hyperspectral image classification. *IEEE Trans. Geosci. Remote Sens.* **2020**, *59*, 3396–3408. [[CrossRef](#)]
22. Han, K.; Wang, Y.; Tian, Q.; Guo, J.; Xu, C.; Xu, C. Ghostnet: More features from cheap operations. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Seattle, WA, USA, 13–19 June 2020; pp. 1580–1589.
23. Wang, C.Y.; Bochkovskiy, A.; Liao, H.Y.M. YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Vancouver, BC, Canada, 17–24 June 2023; pp. 7464–7475.
24. Ye, C.; Guo, Z.; Gao, Y.; Yuan, F.; Wu, M.; Xiao, K. Deep Learning-Driven Body-Sensing Game Action Recognition: A Research on Human Detection Methods Based on MediaPipe and YOLO. In Proceedings of the 2025 6th International Conference on Computer Engineering and Application (ICCEA), Hangzhou, China, 25–27 April 2025; pp. 2087–2092.
25. Zhang, K.; Yuan, F.; Jiang, Y.; Mao, Z.; Zuo, Z.; Peng, Y. A Particle Swarm Optimization-Guided Ivy Algorithm for Global Optimization Problems. *Biomimetics* **2025**, *10*, 342. [[CrossRef](#)]
26. Zhao, Z.; Chen, S.; Ge, Y.; Yang, P.; Wang, Y.; Song, Y. Rt-detr-tomato: Tomato target detection algorithm based on improved rt-detr for agricultural safety production. *Appl. Sci.* **2024**, *14*, 6287. [[CrossRef](#)]
27. Qu, Y.; Wan, B.; Wang, C.; Ju, H.; Yu, J.; Kong, Y.; Chen, X. Optimization algorithm for steel surface defect detection based on PP-YOLOE. *Electronics* **2023**, *12*, 4161. [[CrossRef](#)]
28. Feng, Y.; Huang, J.; Du, S.; Ying, S.; Yong, J.-H.; Li, Y.; Ding, G.; Ji, R.; Gao, Y. Hyper-yolo: When visual object detection meets hypergraph computation. *IEEE Trans. Pattern Anal. Mach. Intell.* **2024**, *47*, 2388–2401. [[CrossRef](#)] [[PubMed](#)]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.